



American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Natural Language Processing

Mid-Term Project Report

Summer 2025-2026

Section: D

Group: 5

Project Contributions

SL #	Student Name & ID	Contributions
1.	Name: Akib Hasan Pyil ID: 23-50531-1	Implemented Bag-of-Words and TF-IDF vectorization (Task 5) and the Naive Bayes classification experiments (Task 6). Wrote the results table and its discussion. Integrated all group members' code into a single working notebook and reran the full pipeline end to end.
2.	Name: Mehjabin Mostafa ID: 23-50707-1	Collected both datasets, Implemented data loading for both datasets (Task 1) and regex-based text cleaning (Task 2). Wrote the Dataset Description section.
3.	Name: Arpita Islam ID: 23-50712-1	Implemented tokenization, case folding, stop word removal and lemmatization (Task 3), and implemented the Logistic Regression classification experiments (Task 7)
4.	Name: Tauhid Sarker ID: 23-52773-2	Did train-test split (Task 4) and the Support Vector Machine classification experiments (Task 8). Compiled and formatted the final report.

Dataset Description

Two publicly available Kaggle datasets on fake news detection are used. Dataset 1 is the ISOT Fake and Real News Dataset, split across Fake.csv and True.csv, with title, text, subject and date columns; a label column is added on load (1 = fake, 0 = real) and the two files are merged into a single dataframe of 44,898 articles. Dataset 2 is the WELFake dataset, a single file (WelFake_Dataset.csv) of 72,095 articles that already carries title, text and label columns, built by its original authors from four merged sources (Kaggle, McIntire, Reuters and BuzzFeed Political) to reduce over-fitting to any single outlet's writing style.

Both datasets are close to balanced (roughly 52% fake / 48% real for Dataset 1, and 51% fake / 49% real for Dataset 2), which is why accuracy is treated as a meaningful metric alongside precision, recall and F1 throughout this project. An 80/20 stratified split is used for both datasets, giving 35,918 training and 8,980 test articles for Dataset 1, and 57,676 training and 14,419 test articles for Dataset 2.

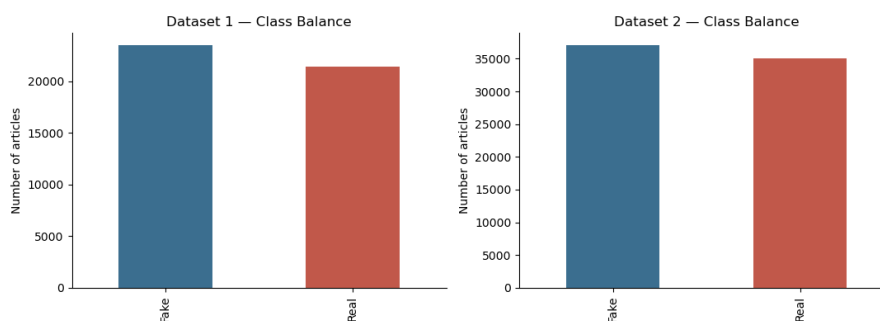


Figure 1: Class balance for Dataset 1 (left) and Dataset 2 (right).

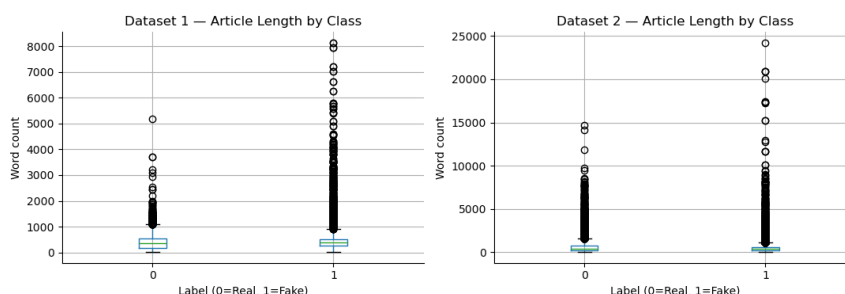


Figure 2: Article length (word count) by class, Dataset 1 vs. Dataset 2.

Project Implementation Detail

Task 1: Data Loading

Dataset 1 is loaded from two separate files, Fake.csv and True.csv, since the raw files do not carry a label column; a label is assigned based on which file each row came from before the two are concatenated into one dataframe. Dataset 2 is loaded from a single file, WelFake_Dataset.csv, which already has a label column, so it only needs column-name cleanup and missing-value handling. In both cases, the title and text columns are joined into a single content column, since a fake news article's headline and body are both informative for classification.

Code for Task 1:

```
fake_df = pd.read_csv("Fake.csv")
true_df = pd.read_csv("True.csv")

fake_df["label"] = 1
true_df["label"] = 0

df1 = pd.concat([fake_df, true_df], ignore_index=True)
df1 = df1.dropna(subset=["title", "text"]).reset_index(drop=True)
df1 = df1.sample(frac=1, random_state=42).reset_index(drop=True)

df1["content"] = df1["title"].astype(str) + " " + df1["text"].astype(str)

df2 = pd.read_csv("WelFake_Dataset.csv")
df2.columns = [c.strip().lower() for c in df2.columns]
if "unnamed: 0" in df2.columns:
    df2 = df2.drop(columns=["unnamed: 0"])

df2["title"] = df2["title"].fillna("")
df2 = df2.dropna(subset=["text", "label"]).reset_index(drop=True)
df2["label"] = df2["label"].astype(int)
```

Output of Task 1:

```
title    Ben Stein Calls Out 9th Circuit Court: Committ...
text     21st Century Wire says Ben Stein, reputable pr...
subject                                     US_News
date                                February 13, 2017
label                                           1

title    Trump Tweets Deranged, Self-Congratulatory 'F...
text     Trump s ongoing meltdown over fake news (the...
label                                           1
```

Code Description of Task 1:

fake_df and true_df are labelled 1 and 0 respectively before being concatenated with pd.concat(), and rows with a missing title or text are dropped since they carry no usable signal. df1.sample(frac=1, random_state=42) shuffles the merged rows so the fake and real articles, which were concatenated in two separate blocks, are not still grouped together before the train-test split later on. For Dataset 2, column names are lowercased and stripped so that later code does not have to special-case capitalisation, and any stray unnamed index column left over from how the CSV was exported is dropped.

Task 2: Text Cleaning with Regular Expressions

Both raw datasets contain noise that regular expressions are well suited to remove URLs, HTML entities, @mentions, #hashtags, and mojibake (character sequences such as â€™ produced when UTF-8 punctuation is decoded as Windows-1252, visible directly in the WELFake sample output above). A single `clean_text` function applies a fixed sequence of `re.sub` substitutions to strip all of this and reduce every document to lowercase-independent letters and whitespace only.

Code for Task 2:

```
url_pattern = re.compile(r"https?://\S+|www\.\S+")
mention_pattern = re.compile(r"@w+")
hashtag_pattern = re.compile(r"#w+")
html_pattern = re.compile(r"<.*?>|&\w+;")
non_letter_pattern = re.compile(r"[^a-zA-Z\s]")
space_pattern = re.compile(r"\s+")

mojibake_map = {
    "\u00e2\u20ac\u2122": "",
    "\u00e2\u20ac\u0153": "",
    "\u00e2\u20ac": "",
}

def clean_text(text):
    text = str(text)
    for bad, good in mojibake_map.items():
        text = text.replace(bad, good)
    text = url_pattern.sub(" ", text)
    text = html_pattern.sub(" ", text)
    text = mention_pattern.sub(" ", text)
    text = hashtag_pattern.sub(" ", text)
    text = non_letter_pattern.sub(" ", text)
    text = space_pattern.sub(" ", text)
    return text.strip()

df1["clean"] = df1["content"].apply(clean_text)
df2["clean"] = df2["content"].apply(clean_text)
```

Output of Task 2:

```
content: Ben Stein Calls Out 9th Circuit Court: Committed a...
clean  : Ben Stein Calls Out th Circuit Court Committed a...

content: Trump drops Steve Bannon from National Security ...
clean  : Trump drops Steve Bannon from National Security...
```

Code Description of Task 2:

The substitutions run in a fixed order inside `clean_text`: mojibake is fixed first with direct string replacement, then `URL_PATTERN` and `HTML_PATTERN` strip links and markup, `MENTION_PATTERN` and `HASHTAG_PATTERN` remove social-media artifacts, and `NON_LETTER_PATTERN` removes everything that is not a letter or whitespace in a single pass, which also removes digits and punctuation at the same time. `SPACE_PATTERN` then collapses the runs of whitespace this leaves behind. Order matters here: URLs have to be removed before the non-letter filter runs, otherwise a URL would just be turned into a long run of stray letters instead of deleted outright.

Task 3: Tokenization, Case Folding, Stop Word Removal and Lemmatization

Once cleaned, each document is tokenized into individual words, lowercased, filtered to remove common English stop words and single-character leftovers, and lemmatized to its dictionary base form. The output is rejoined into a single space-separated string, since scikit-learn's vectorizers in Task 5 expect raw text rather than pre-tokenized lists.

Code for Task 3:

```
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    tokens = word_tokenize(text.lower())
    tokens = [t for t in tokens if t not in stop_words and len(t) > 1]
    tokens = [lemmatizer.lemmatize(t) for t in tokens]
    return " ".join(tokens)

df1["processed"] = df1["clean"].apply(preprocess)
df2["processed"] = df2["clean"].apply(preprocess)
```

Output of Task 3:

```
clean      : Ben Stein Calls Out th Circuit Court Committed...
processed: ben stein call th circuit court committed coup...

clean      : Trump drops Steve Bannon from National Security...
processed: trump drop steve bannon national security coun...
```

Code Description of Task 3:

`word_tokenize(text.lower())` performs case folding and tokenization together, so stop word matching afterward does not have to worry about capitalization. Stop words are removed using NLTK's built-in English stop word list, and tokens of length one (stray leftover letters from Task 2's cleaning) are dropped in the same filter. `WordNetLemmatizer.lemmatize()` then reduces each remaining token to its dictionary form ('Committed' becomes 'committed' after case folding but is not stemmed further, since lemmatization keeps real words rather than truncating them the way a stemmer would). The processed column produced here is what both the BoW and TF-IDF vectorizers in Task 5 are built from.

Task 4: Train-Test Split

Both datasets are split into 80% training and 20% testing partitions using scikit-learn's `train_test_split`, stratified on the label column so the fake/real ratio is preserved in both partitions. The vectorizers in Task 5 fit only on the training partition, so no information from the test set leaks into the vocabulary or the term weightings.

Code for Task 4:

```
train1, test1 = train_test_split(df1, test_size=0.2, stratify=df1["label"], random_state=42)
train2, test2 = train_test_split(df2, test_size=0.2, stratify=df2["label"], random_state=42)

y_train1, y_test1 = train1["label"].values, test1["label"].values
y_train2, y_test2 = train2["label"].values, test2["label"].values
```

Output of Task 4:

```
(35918, 8) (8980, 8)
(57676, 6) (14419, 6)
```

Code Description of Task 4:

`stratify=df1["label"]` is the important argument here: without it, a random split could by chance put disproportionately more fake or real articles into the training set, which would bias every model trained afterward. `random_state=42` is fixed so the split is reproducible across runs. The resulting shapes confirm an 80/20 split on both datasets: 35,918 training and 8,980 test rows for Dataset 1, and 57,676 training and 14,419 test rows for Dataset 2.

Task 5: Bag-of-Words and TF-IDF Vectorization

Each dataset's processed text is converted into two different numeric representations required by the project: Bag-of-Words, using scikit-learn's `CountVectorizer`, and TF-IDF, using `TfidfVectorizer`. Both vectorizers are fit on the training partition only and then used to transform the test partition, for both datasets, giving four feature matrices per dataset in total (two representations, reused by three classifiers each in Tasks 6 through 8).

Code for Task 5:

```
bow_vectorizer1 = CountVectorizer()
X_train1_bow = bow_vectorizer1.fit_transform(train1["processed"])
X_test1_bow = bow_vectorizer1.transform(test1["processed"])

tfidf_vectorizer1 = TfidfVectorizer()
X_train1_tfidf = tfidf_vectorizer1.fit_transform(train1["processed"])
X_test1_tfidf = tfidf_vectorizer1.transform(test1["processed"])

bow_vectorizer2 = CountVectorizer()
X_train2_bow = bow_vectorizer2.fit_transform(train2["processed"])
X_test2_bow = bow_vectorizer2.transform(test2["processed"])

tfidf_vectorizer2 = TfidfVectorizer()
X_train2_tfidf = tfidf_vectorizer2.fit_transform(train2["processed"])
X_test2_tfidf = tfidf_vectorizer2.transform(test2["processed"])
```

Output of Task 5:

```
(35918, 87299) (35918, 87299)
(57676, 160807) (57676, 160807)
```

Code Description of Task 5:

`CountVectorizer().fit_transform()` builds a vocabulary from the training text and returns a sparse matrix of raw word counts per document; `TfidfVectorizer()` builds the same kind of vocabulary but weights each count by inverse document frequency, so words that appear in almost every document contribute less. Both vectorizers are only fit on the training partitions (`fit_transform`) and then applied to the test partitions with `transform` alone, so the vocabulary and IDF weights come entirely from training data. The resulting vocabulary sizes, 87,299 words for Dataset 1 and 160,807 for Dataset 2, are larger than they would be with a minimum document-frequency cutoff, since `CountVectorizer` and `TfidfVectorizer` are used here with their default settings, as required.

Task 6: Naive Bayes Classification

MultinomialNB, the multinomial Naive Bayes classifier from scikit-learn with its default configuration (Laplace smoothing, $\alpha=1.0$), is run against both the Bag-of-Words and TF-IDF representations of both datasets using a shared `run_model` helper (defined once, ahead of Task 6, so it does not need repeating in every classifier task).

Code for Task 6:

```
run_model(MultinomialNB(), X_train1_bow, y_train1, X_test1_bow, y_test1,
          "Naive Bayes", "BoW", "Dataset 1")
run_model(MultinomialNB(), X_train1_tfidf, y_train1, X_test1_tfidf, y_test1,
          "Naive Bayes", "TF-IDF", "Dataset 1")
run_model(MultinomialNB(), X_train2_bow, y_train2, X_test2_bow, y_test2,
          "Naive Bayes", "BoW", "Dataset 2")
run_model(MultinomialNB(), X_train2_tfidf, y_train2, X_test2_tfidf, y_test2,
          "Naive Bayes", "TF-IDF", "Dataset 2")
```

Output of Task 6:

```
Naive Bayes BoW Dataset 1
Accuracy: 0.9543 Precision: 0.9565 Recall: 0.9561 F1: 0.9563
[[4080 204]
 [ 206 4490]]

Naive Bayes TF-IDF Dataset 1
Accuracy: 0.9384 Precision: 0.9335 Recall: 0.95 F1: 0.9416
[[3966 318]
 [ 235 4461]]

Naive Bayes BoW Dataset 2
Accuracy: 0.8991 Precision: 0.8989 Recall: 0.9056 F1: 0.9022
[[6251 755]
 [ 700 6713]]

Naive Bayes TF-IDF Dataset 2
Accuracy: 0.8682 Precision: 0.8566 Recall: 0.893 F1: 0.8744
[[5898 1108]
 [ 793 6620]]
```

Code Description of Task 6:

`run_model()` fits the given classifier on the training features and labels, predicts on the test features, computes accuracy, precision, recall and F1 with scikit-learn's metrics functions, appends the results to a shared results list for the final table, and prints the confusion matrix. The same function is called four times in this task, once per combination of representation and dataset, with only the vectorized inputs and labels changing. Naive Bayes performs noticeably worse with TF-IDF than with Bag-of-Words on both datasets (Dataset 1: 95.43% to 93.84%; Dataset 2: 89.91% to 86.82%) -- multinomial Naive Bayes' probability estimates are derived from raw word counts, so feeding it TF-IDF's down-weighted, non-integer values works somewhat against its own underlying assumptions, unlike the two classifiers in the next two tasks, which do not make that assumption.

Task 7: Logistic Regression Classification

The same `run_model` helper is reused with scikit-learn's `LogisticRegression`, again with its default configuration, on all four representation/dataset combinations.

Code for Task 7:

```
run_model(LogisticRegression(), X_train1_bow, y_train1, X_test1_bow, y_test1,
          "Logistic Regression", "BoW", "Dataset 1")
run_model(LogisticRegression(), X_train1_tfidf, y_train1, X_test1_tfidf, y_test1,
          "Logistic Regression", "TF-IDF", "Dataset 1")
run_model(LogisticRegression(), X_train2_bow, y_train2, X_test2_bow, y_test2,
          "Logistic Regression", "BoW", "Dataset 2")
run_model(LogisticRegression(), X_train2_tfidf, y_train2, X_test2_tfidf, y_test2,
          "Logistic Regression", "TF-IDF", "Dataset 2")
```

Output of Task 7:

```
Logistic Regression BoW Dataset 1
Accuracy: 0.9963 Precision: 0.9966 Recall: 0.9964 F1: 0.9965
[[4268  16]
 [  17 4679]]

Logistic Regression TF-IDF Dataset 1
Accuracy: 0.989 Precision: 0.9923 Recall: 0.9866 F1: 0.9894
[[4248  36]
 [  63 4633]]

Logistic Regression BoW Dataset 2
Accuracy: 0.9621 Precision: 0.9508 Recall: 0.9768 F1: 0.9636
[[6631 375]
 [ 172 7241]]

Logistic Regression TF-IDF Dataset 2
Accuracy: 0.951 Precision: 0.9448 Recall: 0.9607 F1: 0.9527
[[6590 416]
 [ 291 7122]]
```

Code Description of Task 7:

No new logic is needed here: `run_model()` is generic over any scikit-learn classifier that implements `fit` and `predict`, so `LogisticRegression()` is passed in exactly the way `MultinomialNB()` was in Task 6. Logistic Regression is the strongest performer on Dataset 1 (99.63% accuracy with Bag-of-Words), and its confusion matrix shows only 33 misclassifications out of 8,980 test articles, split almost evenly between false positives and false negatives rather than being biased toward one class. Performance drops on Dataset 2 (96.21%), consistent with every classifier in this project scoring lower there, which points to Dataset 2 being the harder of the two datasets rather than Logistic Regression specifically struggling with it.

Task 8: Support Vector Machine Classification

The same `run_model` helper is used a third time with scikit-learn's `LinearSVC`, the linear Support Vector Machine implementation required by the project, again with its default configuration.

Code for Task 8:

```
run_model(LinearSVC(), X_train1_bow, y_train1, X_test1_bow, y_test1,
          "SVM", "Bow", "Dataset 1")
run_model(LinearSVC(), X_train1_tfidf, y_train1, X_test1_tfidf, y_test1,
          "SVM", "TF-IDF", "Dataset 1")
run_model(LinearSVC(), X_train2_bow, y_train2, X_test2_bow, y_test2,
          "SVM", "Bow", "Dataset 2")
run_model(LinearSVC(), X_train2_tfidf, y_train2, X_test2_tfidf, y_test2,
          "SVM", "TF-IDF", "Dataset 2")
```

Output of Task 8:

```
SVM BoW Dataset 1
Accuracy: 0.9961 Precision: 0.996 Recall: 0.9966 F1: 0.9963
[[4265  19]
 [ 16 4680]]

SVM TF-IDF Dataset 1
Accuracy: 0.9955 Precision: 0.996 Recall: 0.9955 F1: 0.9957
[[4265  19]
 [ 21 4675]]

SVM BoW Dataset 2
Accuracy: 0.9542 Precision: 0.943 Recall: 0.9694 F1: 0.956
[[6572  434]
 [ 227 7186]]

SVM TF-IDF Dataset 2
Accuracy: 0.9673 Precision: 0.959 Recall: 0.9781 F1: 0.9685
[[6696  310]
 [ 162 7251]]
```

Code Description of Task 8:

As with Logistic Regression, no new code is required beyond passing `LinearSVC()` into `run_model()`. SVM is competitive with Logistic Regression on Dataset 1 (99.61% with Bag-of-Words) and is the single best-performing combination on Dataset 2 (96.73% with TF-IDF, the only case in this project where TF-IDF outperforms Bag-of-Words for a given classifier on Dataset 2). This suggests the wider, noisier vocabulary of Dataset 2 benefits from TF-IDF's down-weighting of very common words in a way Dataset 1's more uniform ISOT-sourced text does not.

Results and Discussion

Algorithm	Representation	Dataset 1				Dataset 2			
		Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1
Naive Bayes	BoW	0.9543	0.9565	0.9561	0.9563	0.8991	0.8989	0.9056	0.9022
Naive Bayes	TF-IDF	0.9384	0.9335	0.9500	0.9416	0.8682	0.8566	0.8930	0.8744
Logistic Regression	BoW	0.9963	0.9966	0.9964	0.9965	0.9621	0.9508	0.9768	0.9636
Logistic Regression	TF-IDF	0.9890	0.9923	0.9866	0.9894	0.9510	0.9448	0.9607	0.9527
SVM	BoW	0.9961	0.9960	0.9966	0.9963	0.9542	0.9430	0.9694	0.9560
SVM	TF-IDF	0.9955	0.9960	0.9955	0.9957	0.9673	0.9590	0.9781	0.9685

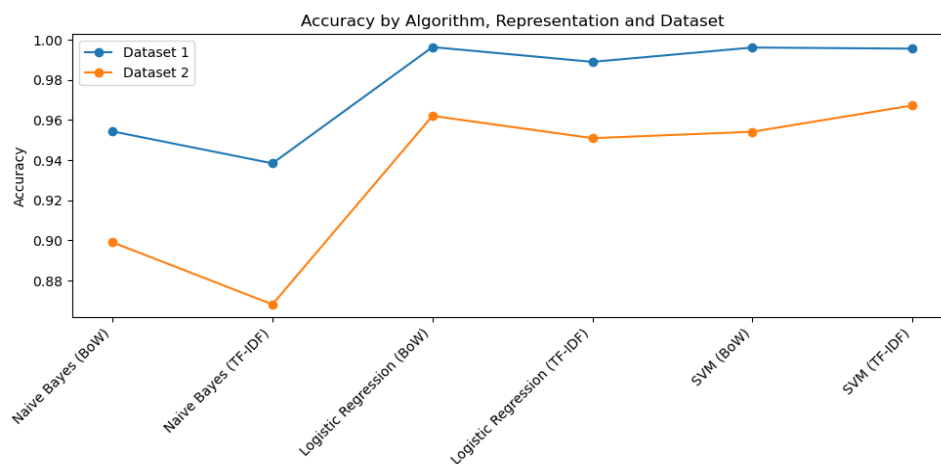


Figure 3: Accuracy across all algorithm/representation combinations, for both datasets.

Across both datasets and both representations, Naive Bayes is consistently the weakest of the three algorithms and is also the only one of the three that performs worse with TF-IDF than with Bag-of-Words (Dataset 1: 95.43% down to 93.84%; Dataset 2: 89.91% down to 86.82%). This is explained by how multinomial Naive Bayes estimates its probabilities: it treats each feature value as a word count, so feeding it TF-IDF's down-weighted, non-integer values works against the assumption the algorithm is built on. Logistic Regression and Support Vector Machine do not share that assumption, so TF-IDF affects them far more mildly, and even helps SVM on Dataset 2 (Bag-of-Words 95.42% to TF-IDF 96.73%).

Every algorithm scores lower on Dataset 2 than on Dataset 1, by roughly 3 to 9 percentage points depending on the algorithm. This is consistent across all six algorithm/representation combinations, which points to Dataset 2 itself being the harder of the two rather than any one classifier struggling with it specifically. Dataset 2 (WELFake) was built by its original authors from four merged sources with different writing styles, whereas Dataset 1 (ISOT) is sourced from a narrower set of outlets, so a less uniform vocabulary and less consistent labelling conventions in Dataset 2 are the most likely explanation.

Logistic Regression and SVM both reach above 99.5% accuracy on Dataset 1 with Bag-of-Words, which is high enough to be worth explaining rather than simply reported: ISOT's real articles are almost all sourced from Reuters and share a highly consistent opening pattern (a dateline such as "WASHINGTON (Reuters) -"), which a linear classifier can key on directly as an almost deterministic signal, independently of the more general fake-versus-real language patterns the project is otherwise measuring. This is specific to Dataset 1's collection process; no equivalent single feature is available on Dataset 2, which is reflected in its consistently lower scores across every model. No single algorithm/representation combination is best on both datasets: Logistic Regression with Bag-of-Words wins on Dataset 1 (99.63%), while SVM with TF-IDF wins on Dataset 2 (96.73%), which is why both datasets and both representations are reported in full above rather than collapsed into one number.

Project Code

The complete, unabridged notebook code, in execution order, is given below:

```
import re
import string
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

warnings.filterwarnings("ignore")

print("Done")

nltk.download("punkt")
nltk.download("punkt_tab")
nltk.download("stopwords")
nltk.download("wordnet")
nltk.download("omw-1.4")

fake_df = pd.read_csv("dataset/Fake.csv")
true_df = pd.read_csv("dataset/True.csv")

fake_df["label"] = 1
true_df["label"] = 0

df1 = pd.concat([fake_df, true_df], ignore_index=True)
df1 = df1.dropna(subset=["title", "text"]).reset_index(drop=True)
df1 = df1.sample(frac=1, random_state=42).reset_index(drop=True)

df1["content"] = df1["title"].astype(str) + " " + df1["text"].astype(str)
df1.head()

df2 = pd.read_csv("dataset/WELFake_Dataset.csv")
df2.columns = [c.strip().lower() for c in df2.columns]
if "unnamed: 0" in df2.columns:
    df2 = df2.drop(columns=["unnamed: 0"])

df2["title"] = df2["title"].fillna("")
df2 = df2.dropna(subset=["text", "label"]).reset_index(drop=True)
df2["label"] = df2["label"].astype(int)
df2 = df2.sample(frac=1, random_state=42).reset_index(drop=True)

df2["content"] = df2["title"].astype(str) + " " + df2["text"].astype(str)
```

```

df2.head()

url_pattern = re.compile(r"https?://\S+|www\.\S+")
mention_pattern = re.compile(r"@w+")
hashtag_pattern = re.compile(r"#w+")
html_pattern = re.compile(r"<.*?>|&\w+;")
non_letter_pattern = re.compile(r"[^a-zA-Z\s]")
space_pattern = re.compile(r"\s+")

mojibake_map = {
    "\u00e2\u20ac\u2122": "",
    "\u00e2\u20ac\u0153": "",
    "\u00e2\u20ac": "",
}

def clean_text(text):
    text = str(text)
    for bad, good in mojibake_map.items():
        text = text.replace(bad, good)
    text = url_pattern.sub(" ", text)
    text = html_pattern.sub(" ", text)
    text = mention_pattern.sub(" ", text)
    text = hashtag_pattern.sub(" ", text)
    text = non_letter_pattern.sub(" ", text)
    text = space_pattern.sub(" ", text)
    return text.strip()

df1["clean"] = df1["content"].apply(clean_text)
df2["clean"] = df2["content"].apply(clean_text)
df1[["content", "clean"]].head(2)

stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    tokens = word_tokenize(text.lower())
    tokens = [t for t in tokens if t not in stop_words and len(t) > 1]
    tokens = [lemmatizer.lemmatize(t) for t in tokens]
    return " ".join(tokens)

df1["processed"] = df1["clean"].apply(preprocess)
df2["processed"] = df2["clean"].apply(preprocess)
df1[["clean", "processed"]].head(2)

train1, test1 = train_test_split(df1, test_size=0.2, stratify=df1["label"], random_state=42)
train2, test2 = train_test_split(df2, test_size=0.2, stratify=df2["label"], random_state=42)

y_train1, y_test1 = train1["label"].values, test1["label"].values
y_train2, y_test2 = train2["label"].values, test2["label"].values

print(train1.shape, test1.shape)
print(train2.shape, test2.shape)

bow_vectorizer1 = CountVectorizer()
X_train1_bow = bow_vectorizer1.fit_transform(train1["processed"])
X_test1_bow = bow_vectorizer1.transform(test1["processed"])

tfidf_vectorizer1 = TfidfVectorizer()
X_train1_tfidf = tfidf_vectorizer1.fit_transform(train1["processed"])
X_test1_tfidf = tfidf_vectorizer1.transform(test1["processed"])

```

```

print(X_train1_bow.shape, X_train1_tfidf.shape)

bow_vectorizer2 = CountVectorizer()
X_train2_bow = bow_vectorizer2.fit_transform(train2["processed"])
X_test2_bow = bow_vectorizer2.transform(test2["processed"])

tfidf_vectorizer2 = TfidfVectorizer()
X_train2_tfidf = tfidf_vectorizer2.fit_transform(train2["processed"])
X_test2_tfidf = tfidf_vectorizer2.transform(test2["processed"])

print(X_train2_bow.shape, X_train2_tfidf.shape)

results = []

def run_model(model, X_train, y_train, X_test, y_test, algo_name, rep_name, dataset_name):
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)
    prec = precision_score(y_test, preds)
    rec = recall_score(y_test, preds)
    f1 = f1_score(y_test, preds)
    results.append({
        "Algorithm": algo_name,
        "Representation": rep_name,
        "Dataset": dataset_name,
        "Accuracy": acc,
        "Precision": prec,
        "Recall": rec,
        "F1": f1,
    })
    print(algo_name, rep_name, dataset_name)
    print("Accuracy:", round(acc, 4), "Precision:", round(prec, 4), "Recall:", round(rec, 4),
"F1:", round(f1, 4))
    print(confusion_matrix(y_test, preds))
    return model

run_model(MultinomialNB(), X_train1_bow, y_train1, X_test1_bow, y_test1, "Naive Bayes", "BoW",
"Dataset 1")

run_model(MultinomialNB(), X_train1_tfidf, y_train1, X_test1_tfidf, y_test1, "Naive Bayes", "TF-
IDF", "Dataset 1")

run_model(LogisticRegression(), X_train1_bow, y_train1, X_test1_bow, y_test1, "Logistic
Regression", "BoW", "Dataset 1")

run_model(LogisticRegression(), X_train1_tfidf, y_train1, X_test1_tfidf, y_test1, "Logistic
Regression", "TF-IDF", "Dataset 1")

run_model(LinearSVC(), X_train1_bow, y_train1, X_test1_bow, y_test1, "SVM", "BoW", "Dataset 1")

run_model(LinearSVC(), X_train1_tfidf, y_train1, X_test1_tfidf, y_test1, "SVM", "TF-IDF",
"Dataset 1")

run_model(MultinomialNB(), X_train2_bow, y_train2, X_test2_bow, y_test2, "Naive Bayes", "BoW",
"Dataset 2")

run_model(MultinomialNB(), X_train2_tfidf, y_train2, X_test2_tfidf, y_test2, "Naive Bayes", "TF-
IDF", "Dataset 2")

```



```

run_model(LogisticRegression(), X_train2_bow, y_train2, X_test2_bow, y_test2, "Logistic
Regression", "BoW", "Dataset 2")

run_model(LogisticRegression(), X_train2_tfidf, y_train2, X_test2_tfidf, y_test2, "Logistic
Regression", "TF-IDF", "Dataset 2")

run_model(LinearSVC(), X_train2_bow, y_train2, X_test2_bow, y_test2, "SVM", "BoW", "Dataset 2")

run_model(LinearSVC(), X_train2_tfidf, y_train2, X_test2_tfidf, y_test2, "SVM", "TF-IDF",
"Dataset 2")

results_df = pd.DataFrame(results)

table = results_df.pivot(index=["Algorithm", "Representation"], columns="Dataset",
                          values=["Accuracy", "Precision", "Recall", "F1"])
table = table.swaplevel(axis=1).sort_index(axis=1, level=0)
table

fig, ax = plt.subplots(figsize=(10, 5))
for dataset in results_df["Dataset"].unique():
    subset = results_df[results_df["Dataset"] == dataset]
    labels = subset["Algorithm"] + " (" + subset["Representation"] + ")"
    ax.plot(labels, subset["Accuracy"], marker="o", label=dataset)

ax.set_ylabel("Accuracy")
ax.set_title("Accuracy by Algorithm, Representation and Dataset")
plt.xticks(rotation=45, ha="right")
ax.legend()
plt.tight_layout()
plt.show()

```